Module 3 코드 안내서 — 파일별 역할과 실행 흐름

> 대상: Module 3 코드를 처음 읽는 사람 (P4 본인 포함) 설계 결정과 근거는 README.md에 있다. 이 문서는 **"코드가 어떤 순서로 실행되고, 각 파일이 그중 어디를 맡는가"** 만 다룬다. 기준 시점: 2026-11-02 (테스트 43개 통과, 예측 모델은 가짜 모델)

---

0. 먼저 큰 그림

Module 3가 하는 일은 한 문장이다.

> **원료 성분을 받아서, 목표 회수율(Ni ≥ 95%, Co ≥ 90%)을 만족하는 조건 중 황산을 가장 적게 쓰는 (황산량, 온도, 시간)을 찾아 레시피로 내보낸다.**

명령 한 줄을 실행하면 아래 순서로 파일들이 차례로 호출된다. 이 문서의 1~9장은 이 순서를 그대로 따라간다.

```
python -m module3_optimization.recommend --batch-id BP-2025-0048
│
├─ 1장  recommend.py          main()              명령줄 인자 해석, 원료 한 건 준비
├─ 2장  config.yaml/config.py                      모든 설정값 읽기
├─ 3장  problem.py            validate_feed()     원료 입력 검사 (결측이면 거절)
├─ 4장  domain.py             build_domains()     "이 원료가 과거 데이터 범위 안인가?"
│                                                   밖이면 → out_of_domain 으로 바로 종료
├─ 5장  predictors.py         build_predictor()   예측 모델 준비 (지금은 가짜 모델)
│       problem.py            Problem             원료 + 설정 + 모델을 하나로 묶음
├─ 6장  optimizers/grid.py    make_grid()         후보 공정 조건 93,240개 생성
│       problem.py            evaluate()          후보 전부를 한 번에 예측 · 판정
│       constraints.py                             제약별 통과 여부 계산
├─ 7장  optimizers/base.py    select_best()       통과한 후보 중 최선 고르기
│                             select_closest()    하나도 없으면 가장 근접한 후보
├─ 8장  recommend.py          _snap_and_verify()  현장 단위 재확인
│                             grade_confidence()  예측 구간 · 신뢰도 · 안내 문구
│       problem.py            Recipe              결과를 정해진 형식에 담음
└─ 9장  report.py             format_card()       레시피 카드 / JSON / CSV 로 출력
```

10장 이후는 이 흐름 바깥에 있는 파일들이다 (일괄 실행, 분석 스크립트, 테스트, 기존 뼈대 파일).

핵심 세 단계 — 5장 · 6장 · 7장

추천의 본체는 이 세 장이다. 헷갈리기 쉬우니 역할을 먼저 구분해 둔다.

장	하는 일	비유
5장	예측 모델을 준비하고, 원료 · 설정과 함께 `Problem` 에 담는다. **아직 아무것도 계산하지 않는다**	계산기 꺼내 두기
6장	(황산, 온도, 시간) 조합을 전부 만들고(93,240개), **한 번에** 예측하고, 조합마다 제약 통과 여부와 비용을 붙인다	모든 경우를 계산기에 넣어 표 만들기

장	하는 일	비유
7장	표에서 통과한 조합 중 **비용이 가장 작은 것**을 고른다	표에서 정답 고르기

데이터셋(원본 엑셀)은 어디서 읽나

5장에서 읽지 않는다. 예측 모델은 추천할 때 데이터셋을 다시 보지 않기 때문이다 (가짜 모델은 수식으로 계산하고, 나중에 쓸 P3 모델은 이미 학습이 끝난 상태로 들어온다). 데이터셋을 읽는 곳은 두 군데뿐이다.

읽는 곳	무엇을 위해	읽는 양
1장 `main()` (`--batch-id`, `--all` 일 때)	추천할 배치의 **원료 성분**을 꺼내려고	해당 배치 한 줄 (또는 전체)
4장 `build_domains()`	적용 범위 판정기에 **"과거 배치가 어디에 모여 있는지"** 를 학습시키려고	과거 배치 전체

`--feed feed.json` 으로 원료를 직접 넣고 적용 범위 판정을 끄면(`--no-domain`), 데이터셋 없이도 추천이 돈다.

자주 나오는 용어

용어	뜻
feed (원료)	주어진 값. Ni·Co·Li·Mn 함량, Cu·Fe·Al 불순물, 원료투입량(`feed_kg`). 바꿀 수 없다
결정 변수	우리가 정하는 값. `acid_l`(황산 L), `temp_c`(온도 ℃), `time_hr`(시간 hr)
후보 (U)	결정 변수 조합 하나. 예: (1010 L, 95 ℃, 8 hr)
μ, σ	예측 평균과 예측 불확실성. 컬럼 `ni_pred`, `ni_std` 등
판정값 (judged)	μ − 1.28σ. 목표 달성 여부는 평균이 아니라 이 보수적인 값으로 본다 (README 3.5)
feasible	모든 제약을 통과한 후보
status	최종 결과 종류. `ok` / `infeasible`(달성 불가) / `out_of_domain`(원료가 범위 밖)

---

1. 시작점 — recommend.py 의 `main()`

역할: 명령줄에서 실행되는 입구. 인자를 읽고 원료 한 건(또는 전체)을 준비해 `recommend()`에 넘긴다.

`main()` 이 하는 일:

1. 인자 해석 — 원료를 어디서 가져올지 셋 중 하나를 고른다
   - `--batch-id BP-2025-0048` : 원본 엑셀에서 해당 배치의 원료 성분을 꺼냄
   - `--feed feed.json` : JSON 파일에 적힌 원료 성분을 사용
   - `--all` : 원본 전체 배치를 한꺼번에 (10장)
2. 옵션 반영 — `--ni-min`/`--co-min`(목표 변경), `--no-domain`(적용 범위 필터 끄기), `--json`(카드 대신 JSON)
3. `feed_from_row()` — 엑셀의 한 행을 원료 dict 로 바꾼다. 빈 칸(NaN)은 `None` 으로 바꿔서 다음 단계에서 "결측"으로 걸리게 한다
4. `recommend()` 호출 → 결과를 9장의 `report` 로 출력

> `recommend()` 가 Module 3 의 **핵심 함수**다. P5 의 API 도 결국 이 함수를 부른다. 2~8장이 모두 이 함수 안에서 일어난다.

---

2. 설정 읽기 — config.yaml, config.py

역할: 코드에 숫자를 직접 쓰지 않기 위해 모든 설정값을 한 곳에 모은다. 값을 바꿀 때는 코드가 아니라 이 파일을 고친다.

`config.yaml` 의 주요 항목:

항목	내용	어디서 쓰이나
`targets`	목표 Ni 95 / Co 90	6장 제약 판정
`uncertainty`	판정용 z = 1.28, 표시용 z = 1.645	6장 판정, 8장 예측 구간
`decision_variables`	각 변수의 범위, 현장 단위(`unit`), 성긴 간격(`coarse_step`)	6장 격자 생성, 박스 제약
`ratio_constraints`	원료 1kg당 황산 2.35 ~ 16.16 L	6장 비율 제약
`cost`	비용 = 황산량 (온도·시간 가중치는 0)	6장 비용 계산
`tie_break`	비용이 같을 때 고르는 순서	7장
`domain`	적용 범위 판정 설정 (켜기/끄기, k, 백분위)	4장
`confidence`	신뢰도 등급 기준	8장
`features`	특성공학 함수 종류 (지금은 아무것도 안 하는 `identity`)	5장
`predictor`	예측 모델 종류와 가짜 모델 상수	5장

`config.py` 의 함수:

- `load_config()` — yaml 을 dict 로 읽는다. `overrides` 를 주면 일부만 덮어쓴다 (테스트에서 "적용 범위 끄기" 같은 변형을 만들 때 사용)
- `get(cfg, "uncertainty.chance_z")` — 점(.)으로 깊은 값을 꺼내는 도우미. 값이 없으면 기본값 반환
- `config_hash()` — 설정 전체를 짧은 문자열로 요약. 결과 JSON 의 `meta.config_hash` 에 들어가서 "어떤 설정으로 나온 결과인지" 추적할 수 있다

---

3. 원료 입력 검사 — problem.py 의 `validate_feed()`

역할: 잘못된 원료 입력을 초기에 막는다.

`validate_feed()`:

- 필수 키 8개(`ni_pct, co_pct, li_pct, mn_pct, cu_pct, fe_pct, al_pct, feed_kg`)가 모두 있는지, 숫자인지, NaN 이 아닌지 검사
- 문제가 있으면 `InputError` 를 던진다. 어떤 키가 빠졌는지(`missing`), 어떤 키가 이상한지(`invalid`)를 담아서
- **결측을 임의로 채우지 않는다.** 대치는 P1(정제) 담당이고, 마음대로 채우면 추천 근거가 왜곡되기 때문
- `batch_id`, `grade` 는 선택 항목으로 그대로 보존

> 원본 420건 중 169건이 이 단계에서 걸린다 (원료 성분에 결측이 있음). P1 정제 데이터가 오면 해결된다.

---

4. 원료가 범위 안인가 — domain.py

역할: 모델이 믿을 만한 영역인지 판정한다 (외삽 방지). 판정기는 두 개이고, 둘 다 **"과거 데이터에서 본 적이 있는가?"**를 서로 다른 대상에 묻는다.

"범위 안"이란 무슨 뜻인가

왜 필요한가. 예측 모델은 과거 배치 데이터만 보고 학습했다. 과거에 비슷한 배치가 많은 곳에서는 예측이 믿을 만하지만, 과거 데이터에서 멀리 떨어진 입력에 대해서는 모델이 "모르는데도" 그럴듯한 숫자를 내놓는다. 이런 예측을 믿고 레시피를 추천하면 현장에서 틀릴 위험이 크다. 그래서 **예측하기 전에 "이 입력 근처에 과거 데이터가 충분히 있는가"를 먼저 확인**한다. 이것을 외삽 방지라고 한다.

"범위"는 컬럼별 최소~최대가 아니다. 여기서 말하는 범위는 "Ni 함량 8~30%" 같은 컬럼별 상·하한이 아니라, **과거 배치들이 실제로 모여 있는 영역**이다. 판정은 "입력과 가장 가까운 과거 배치 5개까지의 거리"로 한다 (4.2).

경우	컬럼별 최소~최대로 보면	이 판정기로 보면
각 값은 평범하지만 **조합이 처음 보는 것** (예: Ni 함량은 높은데 불순물도 매우 높은 원료)	통과	근처에 비슷한 과거 배치가 없으면 **범위 밖**
한 값이 과거 최대보다 **살짝** 큼, 나머지는 흔한 값	탈락	근처에 비슷한 배치가 많으면 **범위 안**
값이 과거 범위를 **크게** 벗어남 (예: Ni 60%)	탈락	**범위 밖**

그러면 데이터셋 범위 안의 데이터만 처리할 수 있는가? 반은 맞다.

- 입력은 무엇이든 받는다. 검사 없이 거절하는 값은 없다 (결측만 3장에서 거절).
- 다만 과거 데이터에서 **많이 떨어진** 입력이면 추천을 **하지 않거나**(원료가 멀면 `out_of_domain`), **그 후보를 빼고** 다른 조건에서 답을 찾는다(공정 조건이 멀면).
- "얼마나 떨어지면 멀다고 볼지"는 과거 배치끼리의 거리 분포로 정한다. 원료는 가장 이상한 1%, 공정 조건은 가장 이상한 5% 수준보다 더 멀면 범위 밖이다 (`config.yaml` 의 `domain`).
- 범위 밖으로 나오는 것은 오류가 아니라 **정직한 답**이다. "이 원료는 과거에 본 적이 없어 예측을 믿을 수 없다"고 알려주고, 성분 재분석이나 소규모 시험을 권한다.

4.1 원료용과 공정용 — 무엇이 다른가

	원료용 (feed)	공정용 (process)
묻는 것	"이 **원료** 자체가 과거에 본 원료들과 비슷한가?"	"이 **원료 + 공정 조건** 조합이 과거에 실제로 해본 조합과 비슷한가?"
비교하는 값	원료 성분 8개 (Ni·Co·Li·Mn 함량, Cu·Fe·Al 불순물, 원료투입량)	위 8개 + 황산·온도·시간 3개 = 11개
검사 횟수	원료당 **한 번**	후보 공정 조건마다 (Grid 기준 93,240번)

	원료용 (feed)	공정용 (process)
걸리면	최적화를 아예 하지 않고 `out_of_domain` 으로 종료	그 후보만 탈락하고 다른 후보로 계속 진행
기준	99 백분위 (`domain.feed_percentile`)	95 백분위 (`domain.percentile`)
쓰이는 곳	이 장 — `recommend()` 초반	6장 — `evaluate()` 안의 `domain` 제약

예시

- **원료용에 걸리는 경우** — 과거 원료의 Ni 함량은 8~30% 였는데 Ni 60% 원료가 들어왔다. 모델은 이런 원료를 학습한 적이 없으니 황산·온도·시간을 어떻게 바꾸든 예측을 믿을 수 없다. 공정 조건을 고르는 것 자체가 의미 없으므로 **처음부터 추천하지 않는다.**
- **공정용에 걸리는 경우** — 원료는 평범한데, 최적화가 "40 ℃ 저온 + 황산 1500 L" 조합을 고르려 한다. 과거에 이런 조합으로 실험한 적이 거의 없다면 그 지점의 예측은 근거가 약하다. 이 **후보만 빼고**, 과거 데이터 근처의 다른 조건 중에서 답을 찾는다.

왜 둘로 나눴나

- **원료는 바꿀 수 없고, 공정 조건은 바꿀 수 있다.** 원료가 이상하면 해결 방법이 없으니 멈추고, 공정 조건이 이상하면 다른 조건을 고르면 된다.
- **기준을 다르게 둘 수 있다.** 원료 하나를 통째로 거절하는 것은 큰 결정이라 느슨하게(99 백분위 — 가장 이상한 1% 만 거절), 후보 하나를 빼는 것은 부담이 적어서 조금 더 엄격하게(95 백분위) 잡았다.

지금 데이터에서 실제로 하는 일

공정용 판정기는 현재 **거의 아무것도 거르지 않는다.** 원본 실험이 황산·온도·시간 범위 전체에 고르게 퍼져 있어서 "해본 적 없는 조합"이 거의 없기 때문이다 (README 7장 이슈 2). 아래 예시에서도 93,240개 후보가 전부 통과한다.

그래서 지금 실질적으로 의미 있는 것은 **원료용 판정기**다. 실제 공장에서 이상한 원료가 들어오거나 성분 분석값을 잘못 입력했을 때 걸러낸다. 공정용은 나중에 데이터 분포가 바뀌거나 결정 변수가 늘어날 때를 대비한 안전장치로 남겨 둔 것이다.

4.2 판정 방법 — `DomainChecker`

두 판정기는 같은 클래스이고, 거리 계산에 쓰는 컬럼과 기준 백분위만 다르다.

1. `fit()` — 학습 데이터를 표준화(평균 0, 표준편차 1)하고, 각 배치에서 가장 가까운 다른 배치 5개까지의 평균 거리를 잰다. 그 거리들의 95(또는 99) 백분위를 **임계값(threshold)** 으로 정한다
2. `distance(X)` — 새 점에서 가장 가까운 학습 배치 5개까지의 평균 거리
3. 거리 ≤ 임계값이면 범위 안
4. `similar_count(X)` — 임계값 반경 안에 있는 학습 배치 수. 레시피 카드의 "유사 과거 배치 N건"

`build_domains()` 가 두 판정기를 만들고, 학습 데이터는 `load_training_data()` 가 설정(`domain.source`)에 따라 가져온다. 지금은 원본 최소 정리본(11장), 나중엔 P1 정제 데이터.

`recommend()` 에서는 (recommend.py#L88) 원료 판정기로 먼저 검사해서, 범위 밖이면 최적화를 아예 하지 않고 `out_of_domain` 결과를 돌려준다.

---

5. 예측 모델과 문제 묶기 — predictors.py, problem.py 의 `Problem`

> **5장에서는 아직 예측이 일어나지 않고, 데이터셋도 읽지 않는다.** 예측 모델이라는 "계산기"를 준비해서 원료·설정과 함께 `Problem` 에 담아 두기만 한다. 계산기에 실제로 값을 넣어 회수율 표를 만드는 것은 6장, 그 표에서 최소 비용 조합을 고르는 것은 7장이다. 데이터셋을 읽는 곳은 1장과 4장뿐이다 (0장 "데이터셋은 어디서 읽나").

5.1 예측 모델 — predictors.py

역할: "원료 + 공정 조건 → 회수율 예측"을 하는 부분. 어떤 모델이든 같은 모양으로 맞춘다.

모든 예측기는 `Predictor` 형태를 따른다:

- 입력: 여러 행짜리 DataFrame (후보 수만 행)
- 출력: 같은 행 수, 컬럼 `ni_pred, co_pred, ni_std, co_std`

클래스	정체	언제 쓰나
`MockAnalyticPredictor`	**가짜 모델.** 수식으로 회수율을 계산. 원본 데이터에 맞춰 상수를 보정함 (11장)	지금 기본값
`ProxyRFPredictor`	**임시 대리 모델.** 결측 없는 원본 행으로 대충 학습한 랜덤 포레스트	실제 모델과 비슷한 성질(계단 모양 예측)을 미리 시험할 때
`RecoveryModelAdapter`	**실제 모델 연결용 변환기.** P3 모델은 (n, 2) 배열을 주므로 이를 위 출력 형태로 바꿔준다. σ 가 없으면 0	P3 모델 도착 후

가짜 모델 수식 (`_recovery`):

```
회수율 = b0 + b_acid·g(원료 1kg당 황산) + b_temp·g(온도 − 40) + b_time·g(시간) − b_imp·불순물 합
g(x) = 1 − exp(−x / 척도)      ← 처음엔 빨리 오르다 점점 둔해지는 곡선
```

황산·온도·시간이 늘면 회수율이 오르기만 하므로(단조 증가), "목표를 만족하는 최소 황산량"을 정확히 계산할 수 있다. 테스트에서 Grid 결과를 이 정답과 비교한다 (12장).

`build_predictor(cfg)` 가 `config.yaml` 의 `predictor.type` 을 보고 위 셋 중 하나를 만든다. **실제 모델로 바꿀 때는 이 설정 한 줄만 바꾼다.**

5.2 문제 묶기 — `Problem`

역할: 최적화에 필요한 것을 한 객체에 담는다. 최적화 알고리즘은 이 객체 하나만 받는다.

담는 것:

- `feed` — 검사를 통과한 원료 (3장)
- `cfg` — 설정 (2장)
- `predictor` — 예측 모델 (5.1)

- `feature_fn` — 특성공학 함수. 지금은 입력을 그대로 돌려주는 `identity_features`, P2 함수가 오면 `build_feature_fn` 이 그걸로 바꿔 끼운다
- `domain` — 공정용 판정기 (4장)
- `target` — 목표. 일부만 주면 그 값만 덮어쓴다

`Problem` 의 가장 중요한 메서드가 다음 장의 `evaluate()` 다. 여기까지는 준비 단계이고, 숫자는 아직 하나도 계산되지 않았다.

---

6. 후보 만들고 한꺼번에 평가하기 — optimizers/grid.py, `Problem.evaluate()`, constraints.py

6.0 한눈에 보기 — 표가 단계별로 어떻게 바뀌나

	5장	**6장**
하는 일	계산기(예측 모델)를 준비하고 `Problem` 에 담는다	계산기에 넣을 **입력(후보)을 만들고**, 실제로 계산해서 **표를 만든다**
결과물	아직 숫자 없음	후보마다 예측값 · 통과 여부 · 비용이 붙은 표

후보란 "우리가 시도해 볼 공정 조건 하나", 즉 (황산, 온도, 시간) 한 세트다. 답을 모르니 가능한 조합을 전부 나열한 뒤, 표 하나로 만들어 단계마다 컬럼을 붙여 나간다. (아래 수치는 `BP-2025-0048` 실행 예. 중간 줄의 예측값은 설명용)

① **후보 만들기** — `make_grid()` (6.1). 조건만 있고 회수율은 아직 없다.

```
후보    황산(L)  온도(℃)  시간(hr)
0       400      40       1.0
1       400      40       1.5
...
93239   1500     95       8.0          ← 111 × 56 × 15 = 93,240 줄
```

② **원료 붙이기** — `_attach_feed()`. 모델은 "원료 + 공정 조건"을 함께 받아야 하므로, 고정된 원료 값을 모든 줄에 복사한다.

```
황산  온도  시간  Ni함량  Co함량  ...  원료kg
400   40    1.0   23.5    11.4    ...  117.2
400   40    1.5   23.5    11.4    ...  117.2
...
```

③ **예측하기** — `predictor.predict()`. 93,240 줄 표를 **통째로 한 번** 모델에 넣으면, 줄마다 예측 회수율이 붙어 나온다.

```
황산  온도  시간  ...  Ni예측  Co예측
400   40    1.0   ...  78.2    70.1
```

```
...
1010  95    8.0    ...  97.8    92.4
```

④ **판정하기** — `_constraints()` → `feasible`, `cost`. 줄마다 "목표를 넘었나? 범위 안인가?"를 검사하고 비용을 붙인다.

```
황산  온도  시간  Ni예측  Co예측  통과(feasible)  비용
400   40    1.0   78.2    70.1    ✗               400
...
1010  95    8.0   97.8    92.4    ✓               1010
```

이 표가 7장으로 넘어가고, 7장은 ✓ 인 줄 중 비용이 가장 작은 것을 고른다. 이 예시에서는 93,240 줄 중 3,038 줄이 ✓ 였고, 그중 1010 L · 95 ℃ · 8 hr 가 선택됐다.

"한 번에"가 왜 중요한가

방식	하는 일	모델 호출
한 줄씩	후보 하나 → 예측 → 판정을 93,240번 반복	93,240번
한 번에 (지금 방식)	93,240줄 표 → 예측 1번 → 판정 1번	**1번**

결과는 같지만 모델은 부를 때마다 준비 비용이 들어서, 한 줄씩 하면 수천 배 느려진다. 지금은 원료 한 건에 약 0.5초가 걸리지만 한 줄씩 하면 몇 분이 걸린다. 그래서 `evaluate()` 는 처음부터 여러 줄을 받도록 만들었고, P3 에게도 "`predict()` 가 여러 줄을 한 번에 받게 해 달라"고 요청하는 것이다.

아래 6.1 ~ 6.3 은 위 ①~④ 를 담당하는 코드의 세부 설명이다.

6.1 후보 만들기 — optimizers/grid.py

`make_grid()` 가 결정 변수의 모든 조합을 만든다.

변수	범위	현장 단위	개수
황산	400 ~ 1500 L	10 L	111
온도	40 ~ 95 ℃	1 ℃	56
시간	1 ~ 8 hr	0.5 hr	15
합계			**93,240**

격자를 처음부터 현장 단위로 만들기 때문에, 나온 답을 현장에서 맞출 수 있는 값으로 따로 반올림할 필요가 없다.

`GridSearch.run()` 은 이 격자를 `problem.evaluate()` 에 **한 번에** 넘기고, 결과에서 최선을 고른다 (7장).

6.2 한꺼번에 평가하기 — `Problem.evaluate()`

이 모듈에서 가장 중요한 함수. 후보 N개를 받아서, 각 후보에 예측·판정·비용을 붙인 표를 돌려준다. 순서:

1. `_attach_feed()` — 모든 후보 행에 원료 성분을 붙인다 (원료는 고정값이므로 같은 값이 반복됨)

2. `feature_fn` — 파생 변수 계산 (지금은 아무것도 안 함)
3. `predictor.predict()` — **예측을 단 한 번** 호출. 93,240행을 한 번에 넣는다. 한 행씩 부르면 수천 배 느려진다
4. 행 위치 맞추기 — P2 함수가 행 번호를 바꿔도 예측이 엉뚱한 행에 붙지 않게 한다
5. 계산 컬럼 추가:
    - `acid_per_feed` = 황산 ÷ 원료투입량
    - `uncertainty` = Ni σ + Co σ (7장 동점 처리용)
    - `ni_judged`, `co_judged` = μ − 1.28σ (보수적 판정값)
    - `shortfall_ni/co/total` = 목표까지 모자란 양 (달성 불가일 때 쓰임)
    - `domain_distance` = 공정용 판정기의 거리
6. `_constraints()` — 6.3 의 제약 함수들을 모두 실행
7. 종합 컬럼:
    - `hard_ok` = **목표를 뺀** 나머지 제약을 모두 통과 (박스·비율·적용 범위)
    - `feasible` = 목표까지 포함해 **모든** 제약 통과
    - `violation` = 위반량 합계
    - `cost` = `cost()`, 지금은 황산량

결과 표의 컬럼 (실제 실행 기준):

```
결정 변수     acid_l, temp_c, time_hr
예측          ni_pred, co_pred, ni_std, co_std
계산          acid_per_feed, uncertainty, ni_judged, co_judged,
              shortfall_ni, shortfall_co, shortfall_total, domain_distance
제약별        ok_box_acid_l / viol_box_acid_l, ok_box_temp_c, ok_box_time_hr,
              ok_ratio_acid_per_feed, ok_domain, ok_target_ni, ok_target_co (+ 각
viol_)
종합          hard_ok, feasible, violation, cost
Grid 가 추가  eval_order
```

6.3 제약 조건 — constraints.py

역할: "이 후보가 조건을 지키는가"를 계산하는 작은 함수들.

모든 제약 함수는 **두 가지를 함께** 돌려준다:

- 통과 여부 (True/False) — Grid 는 이것만 쓴다
- 위반량 (얼마나 벗어났나) — 나중에 GA 는 벌점으로, NSGA-II 는 제약 위반량으로 그대로 쓴다

함수	검사 내용
`bounded()`	기본 함수. lo ≤ 값 ≤ hi (경계 포함, 부동소수점 오차 허용)
`box()`	결정 변수 상·하한
`ratio()`	원료 1kg당 황산 범위
`target()`	판정값 ≥ 목표
`round_to_unit()`	제약은 아니고 도우미. 연속값을 현장 단위로 반올림 (8장에서 사용)

7. 최선의 후보 고르기 — optimizers/base.py, optimizers/__init__.py

역할: 평가된 표에서 답 하나를 고른다. 모든 알고리즘이 같은 규칙을 공유하도록 따로 분리했다.

- `rank_feasible()` — `feasible` 인 행만 남기고 `tie_break` 순서로 정렬
  - 순서: 비용 → 불확실성 → 적용 범위 거리 → 온도 → 시간 (모두 작은 쪽 우선)
  - 표에 없는 컬럼은 건너뛴다 (예: 판정기를 껐을 때 `domain_distance`)
- `select_best()` — 위 정렬의 첫 행. 통과한 후보가 하나도 없으면 `None`
- `select_closest()` — **달성 불가일 때의 대체 답.** `hard_ok` 인 후보 중 ① 목표까지 모자란 양이 가장 작고 ② 그중 비용이 가장 작은 것
- `OptResult` — 알고리즘 실행 결과를 담는 상자 (평가 기록 전체, 최선 해, 평가 횟수, 소요 시간, 알고리즘 이름)
- `Optimizer` — 모든 알고리즘이 지켜야 할 모양: `run(problem, budget, seed) → OptResult`

`get_optimizer("grid")` 는 이름으로 알고리즘을 꺼낸다. 지금 등록된 것은 `grid`(현장 단위)와 `grid_coarse`(성긴 격자, 비교용) 둘이다. 나중에 Random·베이지안·GA 를 여기에 추가한다.

---

8. 결과 다듬기 — recommend.py 후반부, problem.py 의 `Recipe`

최적화가 끝난 뒤 `recommend()` 가 결과를 정리하는 단계.

1. `_snap_and_verify()` — 상위 후보 50개를 현장 단위로 반올림한 뒤 **다시 평가**해서 여전히 통과하는 첫 후보를 고른다. Grid 는 이미 현장 단위라 그대로 통과하지만, 나중에 연속값을 내는 알고리즘(베이지안·GA)을 위한 안전장치다
2. 통과 후보가 있으면 `status = ok`, 없으면 `select_closest()` 로 `status = infeasible`
3. `_interval()` — 예측 구간 μ ± 1.645σ 를 [0, 100] 으로 자름 (회수율은 100% 를 넘을 수 없음)
4. `similar_count()` — 유사 과거 배치 수 (4장)
5. `grade_confidence()` — 신뢰도 등급
   - 달성 불가면 `low`
   - 판정기가 없으면 `medium`
   - σ 가 있고 유사 배치 5건 이상이면 `high`, 2건 이상이면 `medium`, 그 외 `low`
6. `_message()` — 작업자용 안내 문구. 달성 불가일 때 부족분을 알려주고, 불순물 합이 6% 를 넘으면 블렌딩을 권한다
7. 모두 `Recipe` 에 담아 반환

`Recipe` 는 결과의 **공식 형식**이다. `to_dict()` / `to_json()` 이 P5(API·Unity)로 넘어가는 JSON 을 만든다. 형식은 README 4.3 참고.

status	recipe·prediction	shortfall
`ok`	추천 조건	없음
`infeasible`	가장 근접한 조건	목표까지 모자란 양 (%p)
`out_of_domain`	없음	없음

---

9. 출력 — report.py

역할: `Recipe` 를 사람이 읽는 형태나 파일로 바꾼다.

함수	출력
`format_card()`	작업자용 텍스트 카드. status 별로 모양이 다르다 (추천 레시피 / 목표 달성 불가 / 추천 불가)
`write_json()`	JSON 파일 저장
`to_frame()` · `write_csv()`	여러 결과를 표로 펼쳐 CSV 저장. 엑셀에서 한글이 깨지지 않게 저장

가짜 모델이나 임시 RF 로 낸 결과에는 `_dev_notice()` 가 **"개발·검증용 모델 결과"** 경고 줄을 자동으로 붙인다. 실수로 가짜 수치를 실제 결과처럼 쓰는 것을 막기 위함이다.

실제 한 건 따라가기 — `--batch-id BP-2025-0048`

1~9장을 실제 실행 한 번에 대입하면 이렇다.

단계	일어난 일
1. main	원본 엑셀에서 BP-2025-0048 행을 꺼내 원료 dict 로 변환
3. validate_feed	8개 키 모두 정상 → 통과
4. 원료 판정	원료 거리가 임계값 이내 → 계속 진행
6. make_grid	후보 93,240개 생성
6. evaluate	예측 1회 호출. 박스·비율·적용 범위 통과(`hard_ok`) 93,240개 전부, 목표까지 통과(`feasible`) **3,038개**
7. select_best	3,038개 중 황산이 가장 적은 후보 → 1010 L · 95 ℃ · 8 hr
8. 다듬기	반올림 후에도 통과, 예측 구간 계산, 유사 배치 15건, σ 있음 → 신뢰도 **높음**
9. 출력	아래 카드

```
[추천 레시피]  배치 BP-2025-0048
  목표 Ni ≥ 95% · Co ≥ 90%
  황산 1010 L (8.62 L/kg) · 95 ℃ · 8 시간
  예상 Ni 97.8% (90% 구간 95.1~100.0) · Co 92.4% (89.4~95.4)
  신뢰도: 높음 — 유사 과거 배치 15건
  ※ 개발·검증용 모델(mock_analytic) 결과 — 실제 추천 수치가 아님
```

(온도·시간이 항상 최대로 나오는 이유는 비용이 황산량뿐이기 때문이다 — README 7장 이슈 1)

---

10. 여러 건 한꺼번에 — recommend.py 의 `recommend_all()`

`--all` 옵션이 부르는 `recommend_all()`:

- 판정기를 **한 번만** 만들어 모든 배치에 재사용한다 (매번 만들면 느림)
- 배치마다 `recommend()` 호출. 원료 결측(`InputError`)은 건너뛰고 따로 모은다
- 결과는 `report.write_csv()` 로 저장 (기본 `artifacts/recipes.csv`, git 에 올라가지 않음)

현재 원본 기준 251건 처리, 169건 건너뜀, 약 2분.

---

11. 개발용 분석 스크립트 — analysis/

추천 흐름에는 들어가지 않는다. **설계 근거를 만들고 가짜 모델을 맞추기 위한** 보조 코드다.

파일	역할	실행
load_raw.py	원본 엑셀을 최소한만 정리 — 중복 제거, 등급 표기 통일, 센티널(999 등) → 결측. 컬럼명을 영어로 바꾸고 `acid_per_feed`, `impurity_total` 을 추가. **P1 정제를 대신하지 않는다**	다른 파일이 import 해서 사용
facts.py	설계 근거 10개 질문에 대한 요약 수치 출력 (성공 6건, Co 병목, Ni–Co 상관 등). README 7장의 표가 여기서 나왔다	`python -m module3_optimization.analysis.facts`
calibrate_mock.py	가짜 모델 상수를 원본 데이터에 맞춘다. 출력값을 `config.yaml` 의 `predictor.mock` 에 손으로 옮긴다. 처음 수식이 실패한 기록도 파일 상단에 있다	`python -m module3_optimization.analysis.calibrate_mock`

`load_raw.py` 는 4장 판정기 학습 데이터, `--batch-id`·`--all` 의 원료 공급원으로도 쓰인다.

---

12. 테스트 — tests/

`pytest module3_optimization/tests` 로 실행한다 (43개). **원본 엑셀 없이도 통과**하도록 만들었다 — 판정기 테스트는 무작위로 만든 가짜 학습 데이터를 쓴다.

파일	확인하는 것
conftest.py	테스트 공용 준비물. `FEED_A`(불순물 3%, 달성 가능한 원료), `FEED_C`(불순물 16%, 달성 불가 원료), 설정
test_core.py	제약 함수, 가짜 모델 단조성, 어댑터 변환, 입력 검사, 설정 해시, `evaluate()` (1행 = N행 결과, 행 번호 바뀌어도 정상, 확률 제약 계산), `Recipe`
test_grid.py	격자 크기, 현장 단위 답, 최소 비용 선택, 달성 불가 시 근접 해, 동점 처리 순서

파일	확인하는 것
test_domain.py	임계값이 정확히 95 백분위인지, 먼 점 판정, 결측 처리. 가짜 학습 데이터 생성 함수 `synthetic_train()` 도 여기 있다
test_recommend.py	status 3종, 결측 입력 거절, 구간 자르기, 결정성(같은 입력 → 같은 출력), 카드·JSON·CSV, **정답 검증**

정답 검증 (test_recommend.py 의 `test_grid_matches_bisection_optimum`): 가짜 모델은 단조 증가라서, 온도·시간을 고정하면 "목표를 만족하는 최소 황산량"을 이분 탐색으로 정확히 구할 수 있다. Grid 가 찾은 답이 이 정답과 현장 단위(10 L) 안에서 일치하는지 확인한다. 보고서의 "알고리즘 구현 정확성 검증" 근거가 된다.

---

13. 그 밖의 파일

파일	상태	설명
README.md	문서	설계 결정, 인터페이스, 로드맵, 분석 결과, 결정 기록. **"왜 이렇게 했나"** 는 여기
CODE_GUIDE.md	문서	이 문서. **"코드가 어떻게 흐르나"**
optimize.py	기존 뼈대	팀 저장소 뼈대에 원래 있던 진입점. 지금은 `recommend()` 를 그대로 다시 내보내기만 한다. 다른 팀원이 `from module3_optimization.optimize import recommend` 로 써도 되게 남겨둠
multi_objective.py	**미구현**	NSGA-II 다목적 최적화 자리 (보너스). 지금은 import 하면 바로 오류가 나는 상태이니 import 하지 말 것
__init__.py	빈 파일	폴더를 파이썬 패키지로 만드는 표시
optimizers/__init__.py	사용 중	알고리즘 등록부 (7장)
README.pdf	확인 필요	P4 작업에서 만든 파일이 아님

아직 없는 파일 (계획)

README 5장 디렉터리 구성에는 있지만 아직 만들지 않았다.

파일	내용	단계
optimizers/random_search.py	Random Search	5
optimizers/bayesian.py	Optuna 베이지안 최적화	5
optimizers/genetic.py	pymoo 유전 알고리즘	5
compare.py	알고리즘 비교표 자동 생성	5
visualize.py	등고선, 수렴 곡선, 파레토 그림	6

부록 — 무엇을 바꾸려면 어디를 보나

하고 싶은 것	볼 곳
목표치, 범위, 현장 단위 바꾸기	config.yaml
비용에 온도·시간 넣기	config.yaml 의 cost.weights
판정을 덜/더 엄격하게	config.yaml 의 uncertainty.chance_z (README 3.5)
P3 실제 모델 연결	config.yaml 의 predictor.type: real, path → predictors.py 의 RecoveryModelAdapter
P2 특성공학 연결	config.yaml 의 features.type: p2 → problem.py 의 build_feature_fn
P1 정제 데이터로 판정기 학습	config.yaml 의 domain.source: processed, path → domain.py 의 load_training_data
새 제약 추가	constraints.py 에 함수 → problem.py 의 _constraints() 에 등록
새 알고리즘 추가	optimizers/ 에 run() 을 가진 클래스 → optimizers/__init__.py 의 _REGISTRY 에 등록
카드 문구·모양 바꾸기	report.py 의 format_card(), recommend.py 의 _message()
JSON 형식 바꾸기	problem.py 의 Recipe.to_dict() (P5 와 합의 필요)